

TODO 1 and TODO 2, one step at a time

Remaining: N, then the All / Active / Done buttons. Each slide adds a line or two to `src/ToDoApp.jsx` and says what you should see before moving on.

Start from the untouched starter, with `npm run dev` running. TODOs 3 and 4 are yours.

Count what isn't done — and don't store it

IN `TodoApp`: ONE LINE ABOVE THE `return`, ONE LINE IN THE JSX

```
const [newTitle, setNewTitle] = useState('');
const remaining = todos.filter(t => !t.done).length;
...
return <main ...>
  <h1 className="text-3xl font-bold">To-do list</h1>
  <p>Remaining: {remaining}</p>
```

Green is what you add on this step.

WHAT YOU SEE

“Remaining: 2” under the heading. Check off Buy rice and it says 1. Delete a to-do and it's still right.

WHY

`filter` keeps the to-dos that aren't done; `.length` counts them. It is worked out again on every render, so there is nothing to keep in sync. A count in its own `useState` is the one that goes wrong on delete.

`npm test` both Remaining checks pass. That's feature 1 — 12 points.

The idea is workshop drill 3: if you can compute it from state you already have, don't store it.

Five small steps



- After every step the page still works, and you can see what changed before you go on.
- Steps 1–4 add one piece of state and three buttons. The list doesn't change until step 5.
- Step 5 adds no state. That's the idea being graded.

One filter that is 'all', 'active' or 'done' — not showActive and showDone. Two booleans can contradict each other; one value can't.

One piece of state

IN `TodoApp`, NEXT TO THE OTHER `useState` LINES

```
const [newTitle, setNewTitle] = useState('');  
const [filter, setFilter] = useState('all');  
const remaining = todos.filter(t => !t.done).length;
```

Green is what you add on this step.

WHAT YOU SEE

Nothing new. The page looks exactly as it did — that's correct.

WHY

`filter` will be 'all', 'active' or 'done'. Three possible values, one variable. It starts at 'all' because the page starts by showing everything.

```
npm test  nothing new passes yet.
```

Three buttons that do nothing

IN THE JSX, AT THE TODO 2 COMMENT — ABOVE THE `<ul>`

```
    </Panel>

    <div className="flex gap-2">
      <Button variant="outline">All</Button>
      <Button variant="outline">Active</Button>
      <Button variant="outline">Done</Button>
    </div>

    <ul className="space-y-2">
```

Green is what you add on this step.

WHAT YOU SEE

Three buttons in a row above the list. Clicking them does nothing.

WHY

Get the names on screen right first: exactly All, Active and Done. The checks find the buttons by those names, the way a person would.

```
npm test  nothing new passes yet.
```

Make the clicks set the state

THE SAME THREE BUTTONS

```
<div className="flex gap-2">
  <Button variant="outline"
    onClick={() => setFilter('all')}>All</Button>
  <Button variant="outline"
    onClick={() => setFilter('active')}>Active</Button>
  <Button variant="outline"
    onClick={() => setFilter('done')}>Done</Button>
</div>
```

Green is what you add on this step.

WHAT YOU SEE

Still nothing — the state changes, but nothing on the page reads it yet. To watch it, add `<p>filter is: {filter}</p>` for a minute, click around, then delete it.

WHY

`onClick` takes a function. `() =>` `setFilter('active')` is the function; `setFilter('active')` without the arrow would run during render.

`npm test` nothing new passes yet.

Mark the current button

THE SAME THREE BUTTONS — ONE ATTRIBUTE EACH

```
<Button variant="outline"
  aria-pressed={filter === 'all'}
  onClick={() => setFilter('all')}>All</Button>
<Button variant="outline"
  aria-pressed={filter === 'active'}
  onClick={() => setFilter('active')}>Active</Button>
<Button variant="outline"
  aria-pressed={filter === 'done'}
  onClick={() => setFilter('done')}>Done</Button>
```

Green is what you add on this step.

WHAT YOU SEE

Nothing visible by default — `aria-pressed` is for screen readers and tests. To show it too, swap the variant: `variant={filter === 'all' ? 'default' : 'outline'}`.

WHY

`filter === 'active'` is true for exactly one button at a time, because there is only one filter. With two booleans you'd have to keep three buttons in agreement by hand.

npm test “the current filter button is marked with `aria-pressed`” now passes. The list still shows everything.

Compute the list to show

ONE LINE ABOVE THE `return`, ONE WORD IN THE `<ul>`

```
const remaining = todos.filter(t => !t.done).length;
const visible = todos.filter(t =>
  filter === 'all' || (filter === 'done' === t.done);
...
<ul className="space-y-2">
  {visible.map(todo => (
    <TodoItem key={todo.id} todo={todo} ... />
```

Green is what you add on this step.

WHAT YOU SEE

Active shows Buy rice and Wash apples; Done shows Soak beans. With Active on, check off Buy rice: it leaves the list by itself. Remaining still counts the whole list.

WHY

`visible` is worked out from `todos` and filter on every render, so it cannot go stale. Nobody wrote code to remove the checked item — it just isn't in the answer any more.

`npm test` all three filter checks pass. That's feature 2 — 16 points.

It was `todos.map`; now it is `visible.map`. No new `useState` on this slide.

Two features, one new useState

	feature	state added	everything else
TODO 1	Remaining: N	0 new state	computed from todos
TODO 2	All / Active / Done	1 new state: filter	visible computed from todos and filter

- Before every useState, ask: could this be computed from state I already have?
- TODO 3 is the same question again: store the selected to-do's id, and look the to-do up.
- TODO 4 is the one place a second copy is right — a draft the user is typing — and it needs a key.

TODOs 3 and 4, your two tests, and the write-up are yours. npm test tells you where you are.